

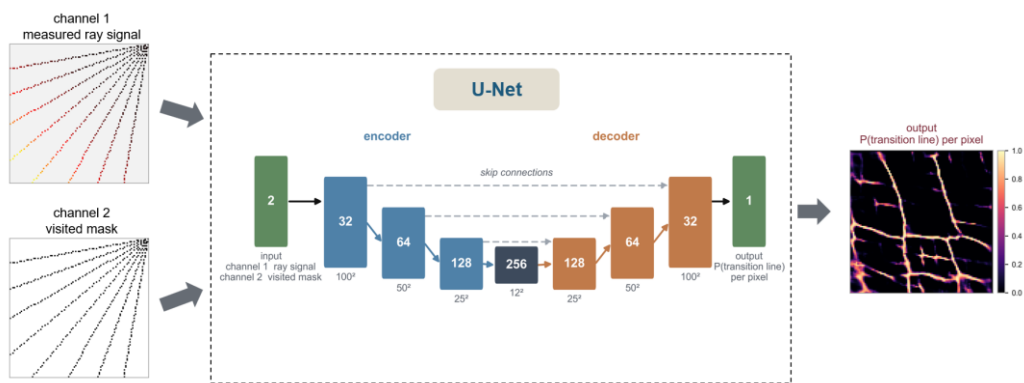
# probabilistic-csd-reconstruction

How the code works: the pipeline, the method it implements, and where every number comes from.  
E. Alizadeh Kashtiban, T. Fujita, A. Oiwa — Osaka University (SANKEN). Generated from the source by docs/codebase/make\_codebase\_doc.py.

## 1. What the code does

The study recovers the charge-transition lines of a simulated double quantum dot from a few percent of the gate-voltage plane. The plane is measured along rays, a U-Net turns what the rays saw into one probability per pixel, and the probability map is cut at a threshold to give transition lines that can be scored against the simulator's exact answer.

Everything hangs on one comparison being fair: two measurement budgets must differ in the measurement and in nothing else. Most of the design below exists to enforce that — one device pool shared by every budget, one split stored with the pool, one architecture with one parameter count, and one set of training constants.



What the network is given and what it returns: two sparse channels in, one probability per pixel out.

The chain, in the order the programs run it:

```
capacitances -> simulated device -> rays -> 2 channels
                                         |
                                         U-Net
                                         |
                                probability per pixel
                                         |
threshold -> transition lines
                                         |
                                F1 at tolerance tau, beside
                                the area that tolerance covers
```

## 2. Layout

**Nothing in src/ has a command line.** The programs are the run\_\*.py files in scripts/: each is a settings block followed by a few calls into the library. That separation is deliberate — a checkpoint is scored by code that cannot have seen the test data, and a re-score can never silently retrain.

| program                   | what it does                                                   |
|---------------------------|----------------------------------------------------------------|
| run_0_full_sweep.py       | THE LAZY BUTTON — steps 1-4 for a whole sweep, in one command. |
| run_1_generate_dataset.py | STEP 1 of 4 — build the dataset for ONE measurement budget.    |

|                                             |                                                                                                               |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>run_2_train_model.py</code>           | STEP 2 of 4 — train the U-Net on one or more configurations.                                                  |
| <code>run_3_evaluate_model.py</code>        | STEP 3 of 4 — score the models on the held-out devices.                                                       |
| <code>run_4_compare_configs.py</code>       | STEP 4 of 4 — every configuration side by side.                                                               |
| <code>run_5_render_device_figures.py</code> | EXTRA — redraw the per-device pictures.                                                                       |
| <code>run_6_threshold_report.py</code>      | EXTRA — show the probability maps and the threshold that turns them into the binary predictions run_3 scores. |
| <code>run_9_update_readme.py</code>         | THE RESULTS SECTION OF README.md — written from the run, not by hand.                                         |
| <code>run_10_bundle_figures.py</code>       | THE GALLERY, SPLIT INTO READABLE BUNDLES.                                                                     |

The library is 6993 lines across 34 modules:

| module                                      | lines | what it is                                                                                                  |
|---------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>                    | 4     | Probabilistic reconstruction of quantum-dot charge stability diagrams from sparse ray measurements.         |
| <code>config/__init__.py</code>             | 19    | (package marker — re-exports only)                                                                          |
| <code>config/axis_labels.py</code>          | 71    | One place that decides what the x and y axes are called.                                                    |
| <code>config/capacitance_config.py</code>   | 286   | The capacitance parameter space every device is drawn from.                                                 |
| <code>config/figure_style.py</code>         | 247   | One place that decides how big every saved figure is.                                                       |
| <code>config/log.py</code>                  | 35    | How much the programs say while they run.                                                                   |
| <code>config/paths.py</code>                | 50    | The project's directory layout, in one place.                                                               |
| <code>ml/__init__.py</code>                 | 1     | The U-Net that turns sparse ray measurements into transition lines.                                         |
| <code>ml/grid_dataset.py</code>             | 54    | (ray peaks -> transition-line map) pairs at a given budget.                                                 |
| <code>ml/grid_metrics.py</code>             | 116   | How close is a predicted transition-line map to the truth.                                                  |
| <code>ml/grid_model.py</code>               | 99    | RayToLinesNet: sparse ray traces -> dense transition-line map.                                              |
| <code>ml/grid_train.py</code>               | 235   | Training and checkpointing for RayToLinesNet.                                                               |
| <code>ml/model_report.py</code>             | 218   | What the network is, as a printable table and as files.                                                     |
| <code>ml/ray_peaks.py</code>                | 216   | The measurement, reproduced offline at any budget.                                                          |
| <code>simulation/__init__.py</code>         | 2     | (package marker — re-exports only)                                                                          |
| <code>simulation/device_factory.py</code>   | 270   | Make the simulated devices the study reads.                                                                 |
| <code>simulation/dqd_simulator.py</code>    | 222   | Runs a charge-sensing DQD simulation and saves all outputs.                                                 |
| <code>simulation/matrix_generator.py</code> | 112   | Generates random capacitance matrices from interval specifications.                                         |
| <code>study/__init__.py</code>              | 18    | The four-stage study, as a library.                                                                         |
| <code>study/comparison.py</code>            | 260   | Stage 4: every configuration side by side.                                                                  |
| <code>study/config.py</code>                | 379   | StudyConfig: one configuration of the study, and the folder it owns.                                        |
| <code>study/dataset.py</code>               | 373   | Stage 1: build one configuration's dataset, and the evidence that it is what the paper says it is.          |
| <code>study/device_figures.py</code>        | 647   | The per-device pictures, re-rendered from the stored arrays into data/<config>/figures/<split>/sample_<i>/. |
| <code>study/device_split.py</code>          | 239   | The train/test split, made once, at the level of the DEVICE.                                                |
| <code>study/evaluation.py</code>            | 385   | Stage 3: score the checkpoint on the held-out devices and draw what it predicts.                            |
| <code>study/figure_bundles.py</code>        | 281   | The comparison gallery, split into readable bundles.                                                        |
| <code>study/model_figures.py</code>         | 547   | The comparison gallery: every way of putting the trained models beside each other.                          |
| <code>study/readme_report.py</code>         | 342   | The results section of README.md, written from the run.                                                     |
| <code>study/sampling.py</code>              | 317   | WHERE the measurement points are put, at a fixed budget.                                                    |
| <code>study/sweep.py</code>                 | 237   | Stages 1-4 for a whole grid of measurement budgets.                                                         |
| <code>study/threshold_report.py</code>      | 513   | What the network really outputs, and what the binarisation threshold does to it.                            |

|                           |     |                                                                  |
|---------------------------|-----|------------------------------------------------------------------|
| study/training.py         | 123 | Stage 2: train the U-Net on one configuration.                   |
| visualization/__init__.py | 2   | Drawing measurements and predictions over a diagram.             |
| visualization/overlay.py  | 73  | Overlays scanned cells, peaks, and rays on charge-sensor images. |

### 3. Stage 1 — the devices, the split and the measurement

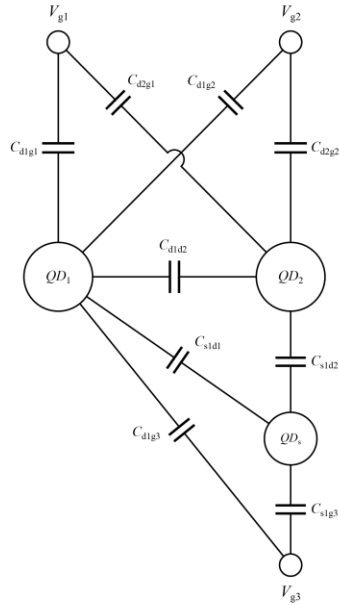
#### The device pool

A device is one uniform draw of 14 capacitances from the intervals in config/capacitance\_config.py, simulated under the constant-capacitance model. **No draw is filtered, inspected or redrawn**: there is no acceptance test anywhere in the pipeline, so the pool is a plain uniform sample of the parameter box and its acceptance rate is 1.00 by construction. A draw is lost only if the simulator itself raises.

The gate-voltage window is randomly offset per device, so the honeycomb is not registered to the image frame and the network cannot learn a fixed position for the lines.

Pools live in data/\_device\_pools/ and are shared by every budget, because devices do not depend on how many rays are fired at them. The folder name carries a fingerprint of the exact capacitance intervals, so changing the parameter space builds a new pool instead of silently reusing the old one.

(a)



(b)

$$\mathbf{C}_{dd} = \begin{bmatrix} C_{d1d1} & C_{d1d2} \\ C_{d2d1} & C_{d2d2} \end{bmatrix}$$

$$\mathbf{C}_{dg} = \begin{bmatrix} C_{d1g1} & C_{d1g2} & C_{d1g3} \\ C_{d2g1} & C_{d2g2} & C_{d2g3} \end{bmatrix}$$

$$\mathbf{C}_{sd} = \begin{bmatrix} C_{s1d1} & C_{s1d2} \end{bmatrix}$$

$$\mathbf{C}_{sg} = \begin{bmatrix} C_{s1g1} & C_{s1g2} & C_{s1g3} \end{bmatrix}$$

The simulated device: the capacitor network and the four capacitance matrices one draw fills in.

#### The split, and the leak it prevents

**The thing that is split is the device, not the image.** Capacitance configurations are drawn first, each gets an ID, and the IDs are split once and written into the pool folder. Every image and every measurement budget inherits the ID of the device it came from.

This matters because a sweep reuses one cached pool across every (rays, points) cell. If each cell re-split, device 37 could train in the 4-ray cell and be held out in the 8-ray cell, and the comparison across cells would stop being like for like. So the split is decided once, outside the sweep, and read back — never recomputed (study/device\_split.py).

separation() then answers the obvious referee question with a number rather than an argument: the smallest Euclidean distance between any training and any test device in normalised parameter space, reported beside the same statistic computed within the training set as a yardstick.

### The measurement

A budget is  $n\_rays \times n\_points$ . The rays fan out from the (max Vx, max Vy) corner at equally spaced angles — `linspace(0, 90, R+2)[1:-1]` degrees — and each is sampled at  $n\_points$  equally spaced positions, assigned to the nearest pixel centre (`ml/ray_peaks.py`).

Because all rays share an origin, points collide near the corner: the number of **distinct** pixels touched is slightly smaller than  $n\_rays \times n\_points$ , and coverage is always quoted as distinct pixels.

The network is handed two channels, both full size:

- `ch0` — the raw sensor value at every pixel a ray visited, zero elsewhere
- `ch1` — a binary visited mask, 1 where any ray passed

A third channel holds the peaks a 1-D peak finder found along each trace. **It is not network input** — it exists for the measurement figures and for the classical baseline.

`ch1` is what separates “measured here, and the signal was low” from “never looked here”. Without it a zero in `ch0` is ambiguous at every pixel and the network cannot tell absence of evidence from evidence of absence.

The input shape is the same for every budget: the budget changes how sparse the two channels are, never how big. One architecture with one parameter count therefore serves the whole sweep, which is what makes a difference across it a difference in measurement.

## 4. Stage 2 — the network and the training

RayToLinesNet (`ml/grid_model.py`) is a U-Net of depth 3, channel widths 32/64/128/256, 1,949,409 trainable parameters. It is fully convolutional — no flatten, no fixed-size layer — so any grid size works and every budget uses the identical network. Upsampling resizes to the skip tensor's own size rather than by a fixed factor, which is what lets an odd ladder (100 → 50 → 25 → 12) reassemble exactly.

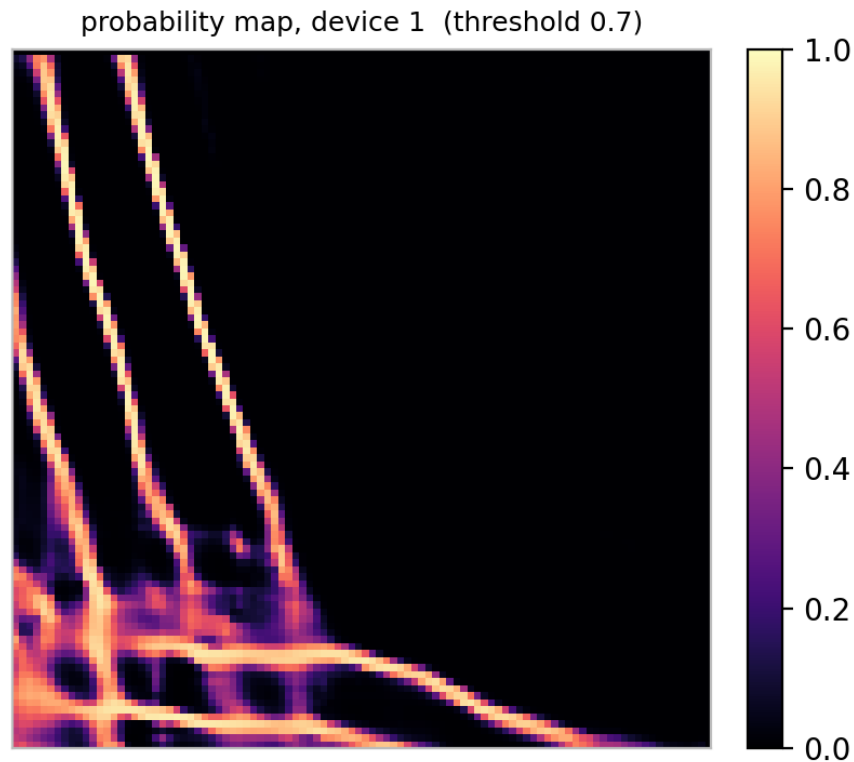
| setting         | value                                                                            | why it is that                                                                                                                           |
|-----------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| loss            | BCEWithLogits (positive class weighted by its rarity, capped at 8.0) + soft Dice | line pixels are a few percent of the diagram, so unweighted BCE converges to “no lines anywhere” and never leaves                        |
| max_pos_weight  | 8.0                                                                              | the raw ~1:30 imbalance makes “call everything a line” the cheapest early minimum, where the network stops depending on its input at all |
| optimiser       | Adam, lr 0.001                                                                   |                                                                                                                                          |
| batch size      | 16                                                                               |                                                                                                                                          |
| val_fraction    | 0.15                                                                             | carved out of the TRAINING devices, never the test set                                                                                   |
| threshold scan  | 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99                                    | scanned on the validation split; the best is stored in the checkpoint                                                                    |
| checkpoint kept | best validation F1@1                                                             | not the last epoch — a run can end on a high loss and still have saved a good model                                                      |

**The test set never enters stage 2.** The validation slice that picks both the epoch and the binarisation threshold is carved out of the training devices, which is what makes stage 3's number an honest held-out result.

## 5. Stage 3 — threshold, tolerance and the score

### Why not 0.5, and why not here

The last layer is a sigmoid, so a prediction is a probability map; every reported number is computed on a binary map cut from it. A fixed 0.5 would be wrong: the loss weights the positive class, which deliberately pushes probabilities up, and the best operating point moves with how sparse the measurement is. The cut is a fitted quantity, so it is chosen out of sample and stored in the checkpoint before any test device is seen.



What the network outputs before thresholding, and where the chosen cut sits.

### Why the score is tolerant

Transition lines are one pixel wide and 7.1 % of the diagram, which breaks the obvious metrics. **Pixel accuracy is useless** — predicting no line anywhere already scores 0.929 — and it is never quoted as a result. Strict pixel F1 is unfairly harsh: a perfectly recovered line displaced by one pixel shares no pixel with the truth and scores zero, although the reconstruction is physically correct.

So the headline is tolerant F1 at  $\tau = 1$  pixel: a predicted pixel counts as correct if a true one lies within  $\tau$ , and a true pixel counts as found if a predicted one lies within  $\tau$ . Distances are Euclidean, so a diagonal neighbour (1.41 px) only counts from  $\tau = 2$ . It is implemented with a distance transform — one EDT per map, not a comparison of every pixel pair.

F1 at  $\tau$  can only rise with  $\tau$ , so it is never quoted alone. Two companions are reported with it:

- **coverage@ $\tau$**  — the fraction of the plane within  $\tau$  pixels of a MEASURED pixel. It describes the experiment and is the same number whatever the model predicts.
- **claimed@ $\tau$**  — the fraction within  $\tau$  pixels of a PREDICTED pixel. It depends on the model, and it is the price of the tolerance: when the discs swallow half the plane, a high F1@ $\tau$  says very little.

Strict IoU is reported too. It punishes one-pixel-wide structures severely and is included for completeness rather than as a headline.

## 6. Stage 4 — comparing budgets

comparison.py reads the metrics.json each evaluation wrote and puts every configuration in one table, in a folder named after the sweep. The name is a pure function of the sweep, so re-running one updates its folder and two different sweeps can never land in the same place.

Configurations that have not been evaluated are listed as missing rather than dropped, so a half-finished sweep cannot quietly look complete. Every row carries the split mode, the resolution and the training-set size, and the comparison warns when they are not constant across the rows being plotted — two budgets are comparable only if everything except the budget matches.

A run that never left its initial plateau still produces a full set of metrics and is indistinguishable in a table from a budget that is simply too small. It is not: it is a failed fit. converged() marks those on the validation score, and they are reported separately rather than dropped — publishing one unmarked would invite the reader to conclude that more rays made things worse.

The gallery is rendered one bundle at a time (figure\_bundles.py) — 40\_points\_per\_ray/, 50\_points\_per\_ray/, and so on. Fifteen labelled series on one axes exhausts the palette and the labels collide, so the combined version was unreadable long before it was wrong.

## 7. The three seeds

| seed       | decides                         | changing it                                                         |
|------------|---------------------------------|---------------------------------------------------------------------|
| seed       | which devices are drawn         | a different pool, and a different pool folder                       |
| split_seed | which devices are held out      | a different assignment of the SAME devices; it does not redraw them |
| train_seed | initial weights and batch order | the same data, a different place for the optimisation to land       |

train\_seed is the one to be careful with. One run per arm is one roll of the dice, and a gap between two arms cannot be told apart from the spread of that roll. model/ and evaluation/ carry the seed in their folder names, so an arm can be trained at several seeds and all the results survive — compare the clusters, not the single numbers. The validation slice is deliberately pinned to a fixed seed, so every repeat of an arm is scored against the same validation devices.

## 8. What ends up on disk

```
data/_device_pools/devices_n550_res100_c1df7b6bf/  shared, expensive
results/<sweep>/
  <budget>/
    config.json          one folder per configuration
    train.npz  test.npz  the settings, read back by 2 and 3
    dataset_summary.json the measured inputs and the answers
    figures/             split and separation evidence
    model/unet.pt        what the dataset looks like
    evaluation/          weights + the chosen threshold
    comparison.csv       metrics.json, per_device.csv, figures
    figures/<bundle>/    one row per configuration
    model_structure.yaml the gallery
    hyperparameters.yaml what the network is
                       what it was run with
```

One run is one folder. The device pools are deliberately left outside it: they are the expensive part and they do not depend on the budget.

## 9. Running it, and checking it

|                                                       |                              |
|-------------------------------------------------------|------------------------------|
| <code>python scripts/run_0_full_sweep.py</code>       | steps 1-4 for a whole grid   |
| <code>python scripts/run_1_generate_dataset.py</code> | one budget's dataset         |
| <code>python scripts/run_2_train_model.py</code>      | train                        |
| <code>python scripts/run_3_evaluate_model.py</code>   | score (compares after)       |
| <code>python scripts/run_9_update_readme.py</code>    | README results, from the run |
| <code>pytest -q</code>                                | 25 tests, no data needed     |

run\_0 takes every setting it does not own from run\_1's CONFIG and calls the same library functions in the same order, so it cannot produce a different answer than running the stages by hand. Check its FRESH\_START and WIPE\_DEVICE\_POOLS flags before running: together they empty results/ and data/.

After any study run, re-run run\_9 — otherwise the README's results section describes a run that no longer exists.

## 10. Appendix — the public surface, module by module

Generated by walking the source: every top-level class and function whose name does not start with an underscore.

### config/axis\_labels.py

One place that decides what the x and y axes are called.

```
AxisLabels  set_axis_labels  get_axis_labels  x_label  y_label
```

### config/capacitance\_config.py

The capacitance parameter space every device is drawn from.

```
as_bands  bounds  sample  overlaps  split_band  split_by_interval  disjoint_report  format_spec  
fingerprint  CapacitanceConfig
```

### config/figure\_style.py

One place that decides how big every saved figure is.

```
draw_ground_truth_map  FigureStyle  set_figure_style  get_figure_style  figure_size  figure_dpi  
new_map_figure  new_figure  apply_voltage_axes  set_no_legend_dir  save_figure
```

### config/log.py

How much the programs say while they run.

```
say  detail  warn
```

### config/paths.py

The project's directory layout, in one place.

```
set_config_root  config_root  data_path
```

### ml/grid\_dataset.py

(ray peaks -> transition-line map) pairs at a given budget.

```
build
```

### ml/grid\_metrics.py

How close is a predicted transition-line map to the truth.

```
tolerant_f1  coverage_at_tau  claimed_at_tau  iou  evaluate
```

### ml/grid\_model.py

RayToLinesNet: sparse ray traces -> dense transition-line map.

RayToLinesNet

### **ml/grid\_train.py**

Training and checkpointing for RayToLinesNet.

```
training_description device predict best_threshold train save load
```

### **ml/model\_report.py**

What the network is, as a printable table and as files.

```
layer_table format_text build_report to_yaml save_model_report
```

### **ml/ray\_peaks.py**

The measurement, reproduced offline at any budget.

```
Measurement load_grid load_ground_truth voltage_to_pixel fan_angles ray_polyline measure  
to_channels
```

### **simulation/device\_factory.py**

Make the simulated devices the study reads.

```
pool_name prune generate
```

### **simulation/dqd\_simulator.py**

Runs a charge-sensing DQD simulation and saves all outputs.

```
DQDSimulator
```

### **simulation/matrix\_generator.py**

Generates random capacitance matrices from interval specifications.

```
CapacitanceMatrixGenerator
```

### **study/comparison.py**

Stage 4: every configuration side by side.

```
collect sweep_name comparability_warnings run configs_in_order
```

### **study/config.py**

StudyConfig: one configuration of the study, and the folder it owns.

```
StudyConfig existing_configs resolve_configs
```

### **study/dataset.py**

Stage 1: build one configuration's dataset, and the evidence that it is what the paper says it is.

```
make_devices make_devices_interval split_devices sample_dirs_for records_for measure_split  
load_split check_split generation_summary summary_text build
```

### **study/device\_figures.py**

The per-device pictures, re-rendered from the stored arrays into data/<config>/figures/<split>/sample\_<i>/.

```
load_device draw_truth ray_geometry fig_charge_sensor fig_stability_diagram fig_rays  
fig_rays_on_truth fig_measurement fig_ray_traces peaks_per_ray fig_all_rays_peaks_overlay  
fig_ml_measurement fig_summary_total fig_panel render_device normalise_devices render_split  
render_config
```

### **study/device\_split.py**

The train/test split, made once, at the level of the DEVICE.

```
load_or_create vector matrix separation observed_ranges interval_check
```

### **study/evaluation.py**

Stage 3: score the checkpoint on the held-out devices and draw what it predicts.

```
per_device_rows score render_device_panels fig_f1_vs_tolerance fig_probability run
```

### **study/figure\_bundles.py**

The comparison gallery, split into readable bundles.

```
sweep_dirs configs_in best_val_f1 group bundle
```

### **study/model\_figures.py**

The comparison gallery: every way of putting the trained models beside each other.

```
Model build_models fig_f1_vs_tolerance fig_train_size fig_tau_all_metrics fig_f1_and_coverage  
fig_f1_and_coverage_shared fig_tau_normalised fig_tau_band fig_training_loss fig_validation_f1  
render_all
```

### **study/readme\_report.py**

The results section of README.md, written from the run.

```
read_rows latest_run converged copy_figures render update run
```

### **study/sampling.py**

WHERE the measurement points are put, at a fixed budget.

```
describe budget lattice_rc random_rc hcut_lines vcut_lines parallel_diag_lines  
random_ray_lines lines_for measure build
```

### **study/sweep.py**

Stages 1-4 for a whole grid of measurement budgets.

```
configs run_dir wipe plan run
```

### **study/threshold\_report.py**

What the network really outputs, and what the binarisation threshold does to it.

```
threshold_grid scan fig_threshold_choice fig_separation fig_overview fig_threshold_strip  
fig_sample_curves run
```

### **study/training.py**

Stage 2: train the U-Net on one configuration.

```
plot_training_curve train
```

### **visualization/overlay.py**

Overlays scanned cells, peaks, and rays on charge-sensor images.

```
OverlayRenderer
```